

Comment la Petite Souris gère ses milliards de dents avec TimescaleDB

Des séries temporelles à croquer

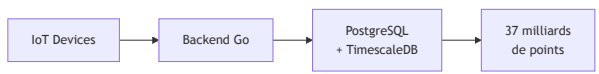
Commencer →

Meetup PostgreSQL Lille - 13 novembre 2025

Qui suis-je ?

David Torres Dias

- 👷 Développeur depuis bientôt **15 ans**
- 🦣 Utilisateur PostgreSQL depuis **10 ans+** (depuis la version 9.4 !)
- 🐹 Amoureux de Go depuis **2015**
- 🚜 DevOps & Dev Backend Go chez **Karnott** depuis **7 ans**
- 📊 Mon histoire avec TimescaleDB a commencé chez Karnott
- GitHub: @defts



C’est quoi TimescaleDB ?

Une extension PostgreSQL

- 📦 **Extension PostgreSQL** pour les séries temporelles
- ✅ **100% compatible** (juste `CREATE EXTENSION`)
- 🔓 **Open source** (Apache 2.0 / TSL)
- 🚀 Utilisé par Cloudflare, le CERN, Karnott…

Le concept clé : Hypertables

```
CREATE TABLE metrics (time TIMESTAMPTZ, device_id INT, value FLOAT);
SELECT create_hypertable('metrics', 'time');
```

→ 📦 Partitionnement automatique • 🔌 API PostgreSQL • ⚡ Optimisations

Mais pour vous montrer concrètement…

On va parler de la **Petite Souris** ! 🐭🦷

Le cas de la Petite Souris SA

Les chiffres du business

- **2 milliards** d'enfants dans le monde
- **~20 dents** de lait par enfant
- **Millions** de transactions par nuit
- Pic d'activité entre 6 et 12 ans

Les défis techniques

- Analyser des milliards de transactions
- Requêtes temporelles complexes
- Stockage qui explose
- Besoin de performances en production

Le schéma de données

Tables de référence

```sql
-- 60 pays
CREATE TABLE countries (
    code TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    continent TEXT
);

-- 20 dents de lait
CREATE TABLE teeth (
    tooth_number INTEGER PRIMARY KEY,
    tooth_name TEXT NOT NULL,
    typical_age INTEGER
);

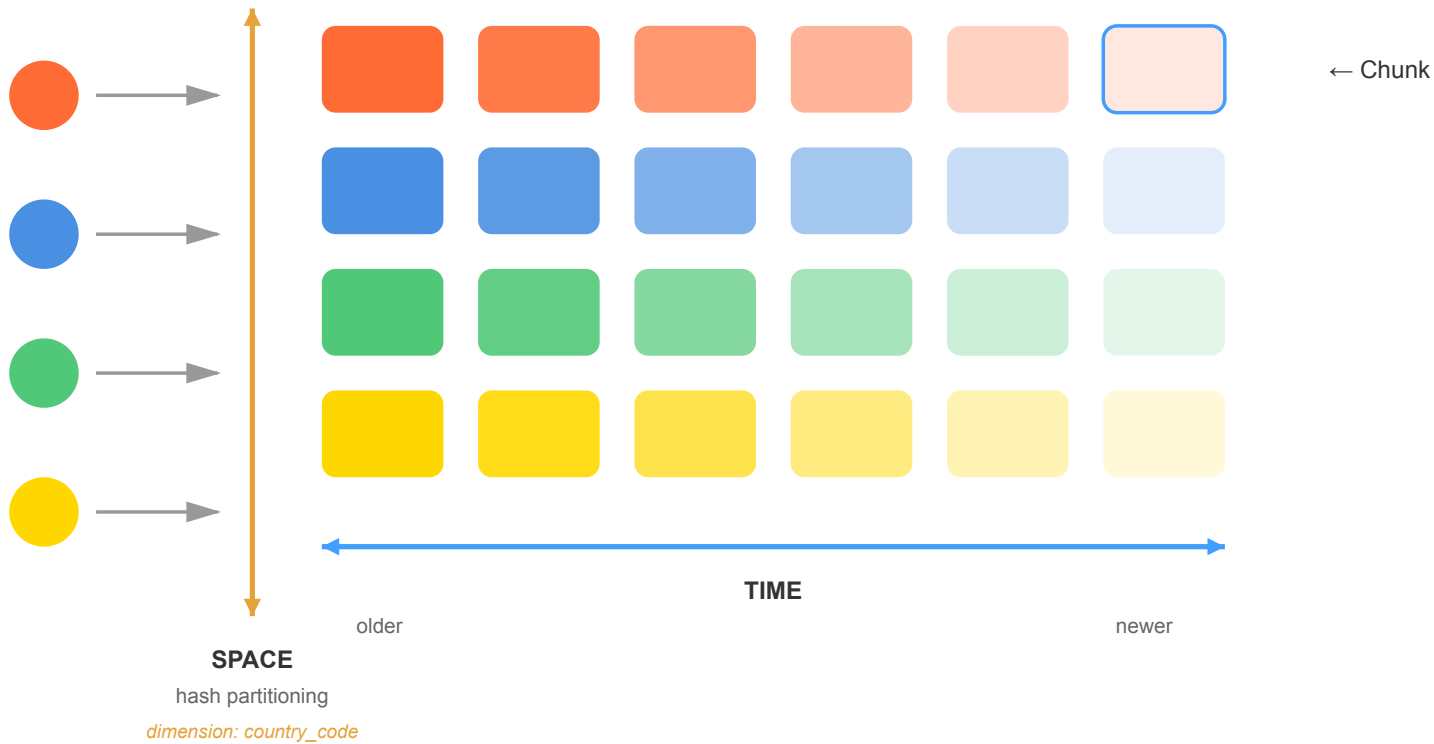
-- 50 millions d'enfants
CREATE TABLE children (
    child_id BIGINT PRIMARY KEY,
    birth_date DATE,
    country_code TEXT REFERENCES countries(code)
);
```

Hypertable

```sql
CREATE TABLE tooth_loss (
    collected_at TIMESTAMPTZ NOT NULL,
    child_id BIGINT
        REFERENCES children(child_id),
    tooth_number INTEGER
        REFERENCES teeth(tooth_number),
    country_code TEXT
        REFERENCES countries(code),
    price DECIMAL(4,2) CHECK (price ⩾ 0)
);

-- Transformer en hypertable
-- Partitionnement temporel + spatial
SELECT create_hypertable(
    'tooth_loss',
    by_range('collected_at', INTERVAL '7 day')
);
SELECT add_dimension(
    'tooth_loss',
    by_hash('country_code', 4)
);
```

Architecture des chunks



La solution : TimescaleDB

Transformons cette table en hypertable et explorons les fonctionnalités !

Au programme



Hyperfunctions

gapfill, stats_agg

Exclusives TimescaleDB



Compression

~95% de réduction

Automatique



Caggs

Pré-calcul

10-100x plus rapide



Retention

Suppression auto

Maintenance zéro

DÉMO : Hyperfunctions TimescaleDB

Ce qu’on va voir

1. 🕳️ **time_bucket_gapfill()** - Remplir les trous dans les séries temporelles
2. 📊 **stats_agg()** - Un scan, toutes les statistiques
3. ⚡ **Function pipelines** (→) - Composition de fonctions lisible
4. 📈 **Percentiles approximatifs** - P50, P95, P99 ultra-rapides

DÉMO : Compression

```
-- depuis TimescaleDB 2.18
-- 1. Activer la compression avec stratégie par pays
ALTER TABLE tooth_loss SET (
    timescaledb.enable_columnstore,
    timescaledb.segmentby = 'country_code'
);

-- 2. Policy : compresser automatiquement les données > 7 jours
SELECT add_columnstore_policy('tooth_loss', INTERVAL '7 days');

-- 3. Compresser manuellement les chunks existants
SELECT convert_to_columnstore(i, if_not_columnstore => true)
FROM show_chunks('tooth_loss', older_than => INTERVAL '7 days') i;

-- avant Timescaledb 2.18
-- ALTER TABLE tooth_loss SET (
--     timescaledb.compress,
--     timescaledb.compress_segmentby = 'country_code'
-- );
-- SELECT add_compression_policy('tooth_loss', INTERVAL '7 days');
-- SELECT compress_chunk(i, if_not_compressed => true)
-- FROM show_chunks('tooth_loss', older_than => INTERVAL '7 days') i;
```

DÉMO : Continuous Aggregates

```sql
-- 1. Créer une vue matérialisée continue avec hyperfunctions + real-time
CREATE MATERIALIZED VIEW daily_revenue_stats
WITH (timescaledb.continuous, timescaledb.materialized_only = false) AS
SELECT
    time_bucket('1 day', collected_at) AS day,
    country_code,
    stats_agg(price) as price_stats,
    percentile_agg(price) as price_percentiles
FROM tooth_loss
GROUP BY day, country_code;

-- 2. Policy de rafraîchissement automatique
SELECT add_continuous_aggregate_policy('daily_revenue_stats',
    start_offset => INTERVAL '1 month',
    end_offset => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour'
);
```

DÉMO : Retention Policies

```sql
-- 1. Voir les données actuelles
SELECT
    COUNT(*) as total_rows,
    MIN(collected_at) as oldest_data,
    MAX(collected_at) as newest_data,
    age(MAX(collected_at), MIN(collected_at)) as data_span
FROM tooth_loss;

-- 2. Ajouter une policy de rétention (1 ligne !)
SELECT add_retention_policy('tooth_loss', INTERVAL '3 years');

-- 3. Voir les policies actives
SELECT * FROM timescaledb_information.jobs
WHERE proc_name = 'policy_retention';
```

🗑️ Les données > 3 ans seront supprimées automatiquement

Retour d’expérience Karnott

Le contexte

- 📍 Agriculture connectée (tracking GPS)
- 📦 **37 milliards** de points de données
- 🚜 Milliers d’engins agricoles
- 📈 **10 hypertables** (location, events, logs, sensors…)

Stack technique

- PostgreSQL 17 + **TimescaleDB 2.18** + PostGIS
- **Google Cloud Platform**
- VM (16 vCPUs, 64 GB RAM)
- Zonal SSD Persistent Disk

Cas d’usage

- 📍 Tracking GPS temps réel
- 🚜 Surfaces travaillées
- 📊 Répartition des travaux
- ⚡ Analyse de productivité
- 🔔 Alertes et anomalies

Défis initiaux

- ❌ Stockage qui explose
- ❌ Coûts GCP qui grimpent

Les gains mesurés avec TimescaleDB

Compression des données

Hypertable	Avant	Après	Gain
device_event	4.6 TB	119 GB	**97.5%**
location	5.9 TB	394 GB	**93.4%**

Moyenne (10 tables) : 94.7% de compression

Impact global

- 💾 **16 TB → 6 TB** • 💰 **10 TB économisés**
- 💰 **-1 800 €/mois** (-21 600 €/an)
- ⚙️ Compression automatique (policies)

→ ROI immédiat sur les coûts cloud !

Points clés à retenir

✅ Les avantages

- Extension PostgreSQL (pas de migration)
- Partitionnement automatique
- time_bucket = agrégations puissantes
- Compression native (~95%)
- Retention automatique
- Continuous aggregates
- Production-ready

⚠️ Pièges à éviter

- Bien choisir la colonne de temps (TIMESTAMPTZ !)
- Adapter la taille des chunks
- Tester la compression avant prod
- Ne pas oublier les index
- Planifier la stratégie de rétention

Ressources

📚 Documentation

- docs.timescale.com
- GitHub TimescaleDB
- Blog Timescale

🎯 Cette présentation

- Repo GitHub : github.com/defts/timescaledb-petite-souris
- Tous les scripts SQL
- Docker Compose prêt à l'emploi
- Documentation complète

Merci ! 🎉

Questions ?

David Torres Dias - david@torresdias.fr

@defts

Présentation créée avec Slidev • github.com/defts/timescaledb-petite-souris